Lecture 6: Markov chain Monte Carlo and Hamiltonian Monte Carlo

COMP 521 • Probability, Optimization, and Scientific Discovery • Section 3.5

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

3.5 Computing posteriors: Markov chain Monte Carlo

Beyond conjugate and Laplace cases, we represent the posterior by *samples*: with draws $\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)}$ any posterior expectation is a Monte Carlo average. Markov chain Monte Carlo (MCMC) builds a chain whose long-run distribution is the posterior, and needs the posterior only up to the unknown constant $p(\mathbf{d})$.

Metropolis. From the current $\boldsymbol{\theta}$, propose $\boldsymbol{\theta}' = \boldsymbol{\theta} + \boldsymbol{\epsilon}$ from a symmetric distribution and accept with probability $\min\{1,\ p(\boldsymbol{\theta}' \mid \mathbf{d})/p(\boldsymbol{\theta} \mid \mathbf{d})\}$; otherwise stay. This satisfies *detailed balance*, $p(\boldsymbol{\theta} \mid \mathbf{d})\, T(\boldsymbol{\theta} \to \boldsymbol{\theta}') = p(\boldsymbol{\theta}' \mid \mathbf{d})\, T(\boldsymbol{\theta}' \to \boldsymbol{\theta})$, where T is the probability of moving between the two states in one step, so the posterior is stationary. The evidence cancels in the ratio. In a long narrow valley, a random-walk proposal must be as small as the valley is narrow, and then it takes many steps to diffuse along the valley.

Hamiltonian Monte Carlo. HMC [1, 2] adds an auxiliary momentum $\mathbf{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$ and simulates the dynamics of a particle with potential energy $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta} \mid \mathbf{d})$,

$$\mathcal{H}(\boldsymbol{\theta}, \mathbf{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\mathbf{p}^{\mathsf{T}}\mathbf{M}^{-1}\mathbf{p}, \qquad \dot{\boldsymbol{\theta}} = \mathbf{M}^{-1}\mathbf{p}, \quad \dot{\mathbf{p}} = -\nabla U(\boldsymbol{\theta}). \tag{29}$$

The *leapfrog* integrator (half kick, drift, half kick) is time-reversible and preserves phase-space volume, so the Metropolis acceptance probability reduces to $\min\{1, e^{-\Delta\mathcal{H}}\}$, with $\Delta\mathcal{H}$ the energy error of the numerical trajectory. A trajectory of tens of steps can cross the whole posterior along the valley while still being accepted with high probability. This is a central reason to make stellar models differentiable: ∇U requires $\nabla_{\boldsymbol{\theta}}\mathcal{G}$. The No-U-Turn Sampler (NUTS) chooses the trajectory length automatically [3].

ex06_hmc.py -- random-walk Metropolis and HMC, from scratch

```python
"""Random-walk Metropolis vs Hamiltonian Monte Carlo, written from scratch in JAX.

Target: a strongly correlated 2-D Gaussian (correlation 0.98), a stand-in for a
posterior with a near-degeneracy between two stellar-physics parameters.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

rho = 0.98
Cov = jnp.array([[1.0, rho], [rho, 1.0]])
Prec = jnp.linalg.inv(Cov)


def log_post(x):  # unnormalized log density
    return -0.5 * x @ Prec @ x


grad_log_post = jax.grad(log_post)


def rwm_step(x, key, step=0.25):
    k1, k2 = jax.random.split(key)
    prop = x + step * jax.random.normal(k1, x.shape)
    accept = jnp.log(jax.random.uniform(k2)) < log_post(prop) - log_post(x)
    return jnp.where(accept, prop, x), accept


def leapfrog(x, p, eps, n_steps):
    p = p + 0.5 * eps * grad_log_post(x)
    def body(_, xp):
        x, p = xp
        x = x + eps * p
        return x, p + eps * grad_log_post(x)
    x, p = jax.lax.fori_loop(0, n_steps - 1, body, (x, p))
    x = x + eps * p
    return x, p + 0.5 * eps * grad_log_post(x)


def hmc_step(x, key, eps=0.15, n_steps=20):
    k1, k2 = jax.random.split(key)
    p0 = jax.random.normal(k1, x.shape)  # identity mass matrix
    x1, p1 = leapfrog(x, p0, eps, n_steps)
    log_r = (log_post(x1) - 0.5 * p1 @ p1) - (log_post(x) - 0.5 * p0 @ p0)  # -dH
    accept = jnp.log(jax.random.uniform(k2)) < log_r
    return jnp.where(accept, x1, x), accept


def run(step_fn, key, n=4000, x0=jnp.array([2.0, -2.0])):
    def body(x, k):
        x, a = step_fn(x, k)
        return x, (x, a)
    _, (xs, acc) = jax.lax.scan(body, x0, jax.random.split(key, n))
    return xs, acc.mean()


```

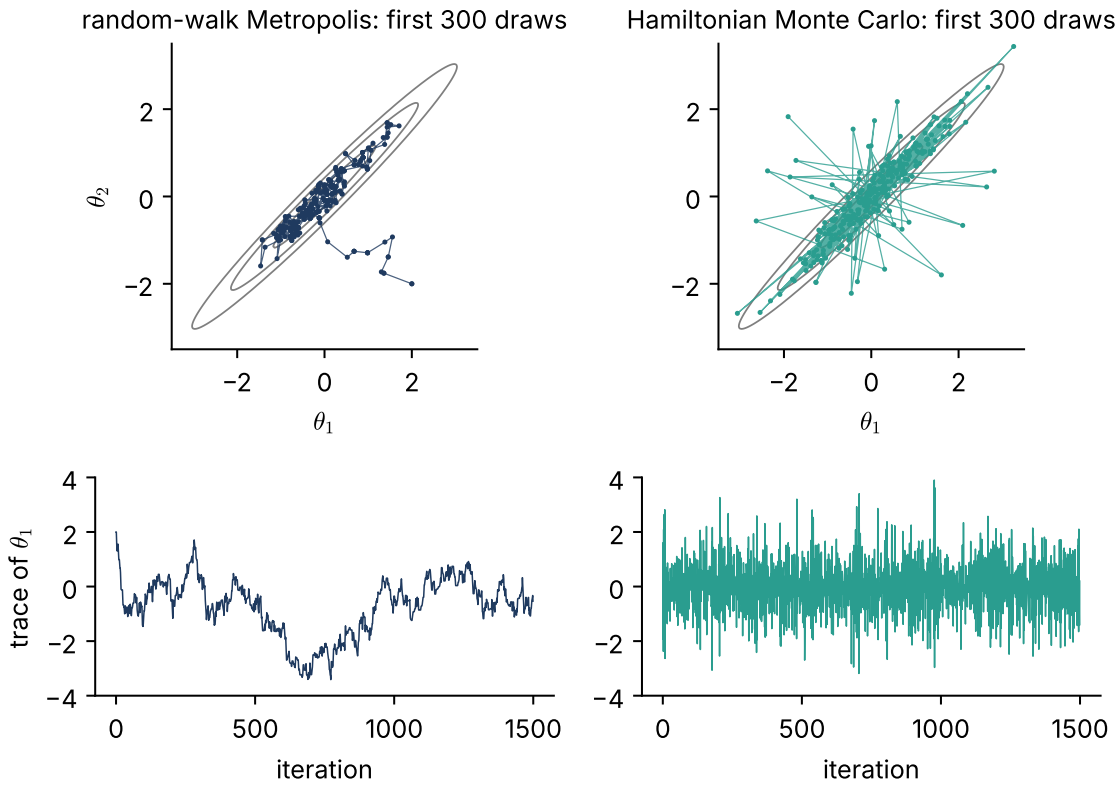


Figure 5: Sampling a correlation-0.98 Gaussian, a stand-in for a near-degenerate pair of physics parameters. Top: the first 300 draws. Bottom: traces. Random-walk Metropolis (left) diffuses slowly along the valley; HMC (right) crosses it in one trajectory.

```python
def ess(chain):
    """Effective sample size with Geyer's initial-positive-sequence truncation."""
    c = chain - chain.mean()
    acf = jnp.correlate(c, c, mode="full")[c.size - 1:] / (c @ c)
    pairs = acf[0:-1:2] + acf[1::2]  # Gamma_k = rho_{2k} + rho_{2k+1}
    keep = jnp.cumprod(pairs > 0)  # stop at the first non-positive pair
    tau = -1.0 + 2.0 * jnp.sum(pairs * keep)  # integrated autocorrelation time
    return chain.size / tau


for name, fn in [("RWM", rwm_step), ("HMC", hmc_step)]:
    xs, acc = run(jax.jit(fn), jax.random.key(7))
    xs = xs[500:]  # discard warm-up
    print(f"{name}: acceptance {acc:.2f}, mean {xs.mean(0)}, sd {xs.std(0)},"
          f" ESS(x1) ~ {ess(xs[:, 0]):.0f} of {xs.shape[0]}")
```

Output (executed for these notes):

```
RWM: acceptance 0.55, mean [-0.30896375 -0.31172623], sd [1.04482784 1.04035183], ESS(x1) ~ 14 of 3500
HMC: acceptance 0.96, mean [-0.0109746  -0.01198857], sd [0.98852799 0.9932576 ], ESS(x1) ~ 10112 of 3500
```

Reading sampler output. Successive MCMC draws are correlated, so the standard error of a posterior mean is $\sigma/\sqrt{N_{\text{eff}}}$ with *effective sample size* $N_{\text{eff}} = N/\tau_{\text{int}}$ and $\tau_{\text{int}} = 1 + 2\sum_{k\geq 1}\rho_k$ the integrated autocorrelation time. The listing estimates τ_{int} by summing autocorrelations in pairs until

a pair turns negative (Geyer's initial positive sequence). Random-walk Metropolis delivers about 14 effective draws from 3500, which is why its mean is off by 0.3: the target has unit standard deviation, so the standard error is $1/\sqrt{14} \approx 0.27$ and the error is about 1σ. HMC reports $N_{\text{eff}} \approx 10^4 > N$. That is not a bug: its successive draws are *anti*-correlated (lag-1 autocorrelation ≈ -0.5), and anticorrelated draws estimate a mean better than independent ones.

Always check, in this order:

1. **Multiple chains from dispersed starts** agree: the rank-normalized split-$\hat{R}$ is below about 1.01 [4].
2. **Effective sample size** is adequate for every quantity you report, including tail quantiles, which mix more slowly than means.
3. **Divergences** (HMC trajectories whose energy error explodes) are absent. They signal regions of high curvature, typically a funnel in a hierarchical model, that the sampler cannot explore.
4. **Coordinates:** if you sample $z = \log \theta$ but your prior is written in θ, add $\log |\mathrm{d}\theta/\mathrm{d}z| = z$ to the log density.

In practice use a tested implementation, for example NUTS in NumPyro or BlackJAX, with ArviZ-style diagnostics. Write the from-scratch version once, so you know what the library is doing when it warns you.

Variational inference instead fits a tractable distribution $q_{\boldsymbol{\omega}}(\boldsymbol{\theta})$ by maximizing a lower bound on $\log p(\mathbf{d})$. It is fast and scales to many parameters, but standard choices underestimate posterior width and miss multimodality. Treat its uncertainties as approximate until checked by the methods of Section 3.7.

Exercise for this lecture

Exercise 12, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

12. [C] Tuning HMC, and a familiar stability limit. In Listing 4, run HMC with step sizes $\varepsilon \in \{0.05, 0.15, 0.25, 0.30, 0.45\}$ at 20 leapfrog steps. Record acceptance rate and ESS. Explain the collapse you observe using the eigenvalues of the target's precision matrix, and connect it to the step-size limit of Section 2.4. Why is the smallest step inefficient although it is almost always accepted?

References

[1] R. M. Neal. MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones, and X.-L. Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman & Hall/CRC, 2011.

[2] M. Betancourt. A conceptual introduction to Hamiltonian Monte Carlo. arXiv:1701.02434, 2017.

[3] M. D. Hoffman and A. Gelman. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15:1593–1623, 2014.

[4] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2):667–718, 2021.